

CSE 4345: Big Data Analytics

Week 6, Part 2 — Seminal Research & Case Study

Department of Computer Science & Engineering
Premier University, Chittagong

Semester 2026

Today's Agenda

Seminal Papers Covered

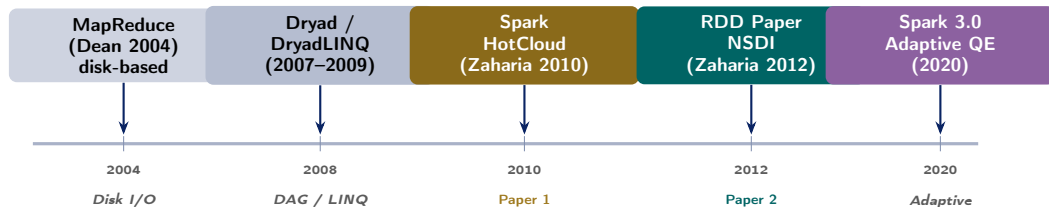
- 1 Zaharia, M., et al. (2010). *Spark: Cluster Computing with Working Sets*. USENIX HotCloud 2010.
- 2 Zaharia, M., et al. (2012). *Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing*. USENIX NSDI 2012.

Case Study

Databricks Daytona GraySort (2013): 100 TB sorted in **23 minutes** on 206 EC2 nodes — 3× faster than Hadoop on 3.5× fewer machines.

Paper 1 — Zaharia et al. (2010): Spark HotCloud

Research Landscape: In-Memory Cluster Computing



The Fundamental Problem with MapReduce

Machine learning and graph algorithms are **iterative**: each iteration reads the output of the previous one. MapReduce writes every intermediate result to HDFS. For k iterations: $k \times$ HDFS read + $k \times$ HDFS write. On spinning disk at 100 MB/s, 10 iterations over 100 GB costs $\approx$ **3 hours of I/O alone**. Spark loads the dataset into RAM *once* and iterates in memory.

Zaharia et al. (2010): HotCloud Publication Context

Full Citation

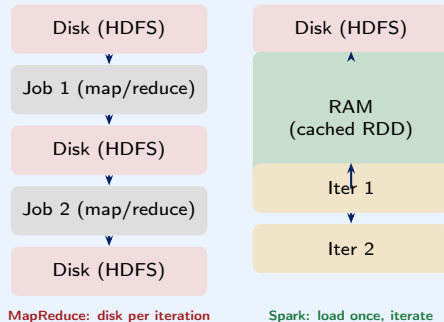
Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., & Stoica, I. (2010). **Spark: Cluster Computing with Working Sets**. In *Proc. 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud '10)*. Boston, MA, USA.

Venue: USENIX HotCloud (workshop, short paper)

Cited by: >5,000 (Google Scholar, 2024)

Authors: Matei Zaharia (PhD student, UC Berkeley AMP Lab), co-advised by Scott Shenker & Ion Stoica

Working Set vs. Streaming



The Key Observation

Many real workloads operate on a **working set** — a dataset that is *reused across multiple parallel operations*:

Paper 2 — Zaharia et al. (2012): RDDs at NSDI

Full Citation

Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., & Stoica, I. (2012). **Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing.** In *Proc. 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI '12)*, pp. 15–28. San Jose, CA, USA.

Venue: USENIX NSDI — Rank A* (CORE), top systems venue

Cited by: >13,000 (Google Scholar, 2024)

Best Paper Award: NSDI 2012

The RDD Definition

*“A **Resilient Distributed Dataset** is a read-only, partitioned collection of records that can be created through deterministic operations on data in stable storage or other RDDs.”*

- **Resilient:** lost partitions are recomputed from lineage, not replicated checkpoints
- **Distributed:** partitions spread across nodes
- **Dataset:** a collection of records, not a mutable key-value store

From Workshop to Full Paper

The 2010 HotCloud paper was a **4-page**

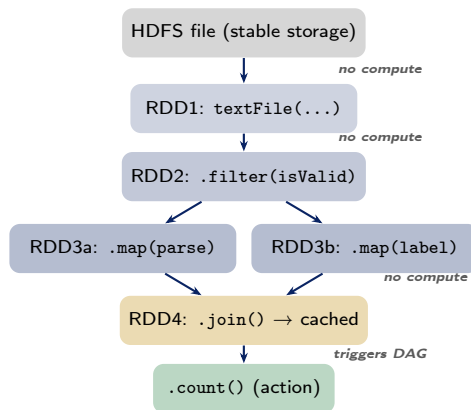
Technical Contribution 1 — RDD Lineage & Fault Tolerance

Lineage Graph: The Key Innovation

RDDs do **not replicate data** to tolerate failures. Instead, each RDD records a **lineage**: a log of the transformations that produced it. If a partition is lost, Spark re-executes only those transformations on the relevant input partitions.

Two types of operations:

- **Transformations** (lazy): `map`, `filter`, `flatMap`, `groupByKey`, `reduceByKey`, `join` — return a new RDD, do *not* trigger computation
- **Actions** (eager): `count`, `collect`, `reduce`, `saveAsTextFile` — trigger a DAG execution from root to output



Technical Contribution 2 — Narrow vs. Wide Dependencies

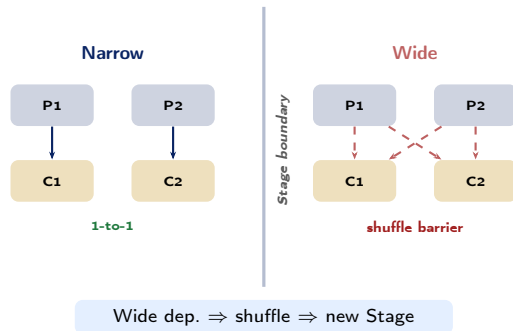
Dependency Classification

Narrow dependency: each parent partition contributes to *at most one* child partition.

- Examples: map, filter, union
- No data shuffle required
- Lost partition recomputed from **one** parent partition
- Pipelined within a single **Stage** (no barrier)

Wide dependency: each parent partition may contribute to *multiple* child partitions.

- Examples: groupByKey, reduceByKey, sortByKey, join (on un-partitioned RDDs)
- Requires a **shuffle** — network-intensive barrier
- Marks a Stage boundary in the DAG



Technical Contribution 3 — Persistence & Partitioning

Persistence Levels (from the paper)

StorageLevel	Meaning
MEMORY_ONLY	JVM heap; spill re-computes
MEMORY_AND_DISK	Heap; overflow to local disk
DISK_ONLY	Local disk; slower reuse
MEMORY_ONLY_SER	Kryo-serialised bytes in heap; smaller but slower
OFF_HEAP	Native memory (Unsafe); avoids GC pressure

Partitioning for Key-Value RDDs

- **Hash partitioner** (default):
`key.hashCode() % numPartitions`
Uniform for random keys; can hot-spot on skewed data
- **Range partitioner**: samples key distribution, assigns ranges to partitions; used by `sortByKey()`
- **Custom partitioner**: user-supplied; co-partition two RDDs on the same key to eliminate shuffle on join

When to Cache?

Cache an RDD if it is **reused in >1 action**
(iterative ML, interactive queries). Caching

Impact: Benchmark Results (RDD paper)

Logistic regression (10 iterations):

What Spark Built on the RDD Foundation

- **Spark SQL / DataFrames** (2014): RDDs wrapped in a typed columnar schema; **Catalyst optimizer** rewrites logical plans; **Tungsten** generates JVM bytecode for execution
- **Spark Streaming / Structured Streaming** (2013/2016): micro-batch or continuous processing via DStreams (RDDs) and then DataStreamReader (DataFrames)
- **MLlib / ML Pipelines** (2014): iterative ML on cached DataFrames; Pipeline API mirrors scikit-learn's fit/transform
- **GraphX** (2014): RDD-backed property graphs for PageRank, label propagation, connected components

Stanford CS246 / MIT 6.830 Perspective

- Stanford CS246 uses Spark as the reference platform for **graph mining** (PageRank via GraphX) and **recommendation systems** (ALS via MLlib); the RDD lineage model is presented as an instance of **dataflow programming**
- MIT 6.830 benchmarks Spark SQL vs. MonetDB (column store) on TPC-H: Spark SQL is within 2× on analytical queries thanks to Catalyst predicate pushdown — despite being a general engine
- CMU 15-721 covers Tungsten's **whole-stage code generation** as a JIT compiler applied to relational operators — a modern take on query compilation

Case Study — Databricks Daytona GraySort (2013)

The GraySort Benchmark

What is Daytona GraySort?

GraySort (named after Jim Gray, Turing Award 1998) is the **world-record sorting competition** for large-scale data systems. Two categories:

- **Daytona**: real-world rules — commodity hardware, standard sort record format (100-byte records, 10-byte key), fault tolerance required, no code specialisation; the prestigious category
- **Indy**: custom hardware / software optimisations allowed

Benchmark dataset: 100 TB of random 100-byte records (10^{12} records).

Metric: wall-clock time to sort, verify, and write output.

Previous Record (Yahoo!, 2009)

Yahoo! sorted 100 TB on a **3,800-node Hadoop MapReduce** cluster in **173 minutes**. In 2012, a team at Yahoo! improved the Hadoop record to **72 minutes** using 2,100 nodes.

Key bottleneck: MapReduce's sort phase writes intermediate data to disk between the map and reduce stages — twice for each byte (spill + merge). At 100 TB, this is ≈ 200 TB of additional disk I/O.

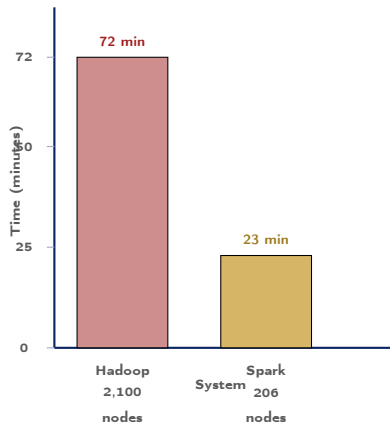
Why Sort?

Sorting exercises every component of a distributed system: network bandwidth, disk

Databricks Result: 100 TB in 23 Minutes (November 2013)

The Databricks Setup

- **Cluster:** 206 EC2 i2.8xlarge instances on Amazon Web Services
- **Per node:** 32 vCPUs, 244 GB RAM, 8×800 GB SSD (local ephemeral storage)
- **Total RAM:** $206 \times 244 = 50.3$ TB
- **Total SSD:** $206 \times 6.4 = 1.3$ PB
- **Network:** 10 Gbps per node
- **Input:** 100 TB of random 100-byte records stored in HDFS (3× replication = 300 TB on disk)
- **Spark version:** 0.8 (pre-DataFrame era)



Result

Key Ratios

- 3.1× faster than Hadoop (72 min → 23 min)

Why Spark Won: Technical Analysis

What Made Spark Faster at GraySort?

- 1 **Sort-based shuffle** (not hash-based): Spark's sort shuffle writes sorted runs to disk once, then merges — MapReduce's hash shuffle spills, merges, and re-reads multiple times. At 100 TB, this eliminates ≈ 120 TB of extra I/O.
- 2 **SSD-aware I/O**: Databricks configured Spark to use all 8 SSDs as shuffle directories simultaneously — $8\times$ the sequential write bandwidth of one SSD.
- 3 **Off-heap sort buffers**: Spark's `UnsafeShuffleWriter` uses `sun.misc.Unsafe` to sort records in native memory, bypassing JVM GC pauses during the sort phase.
- 4 **Fewer stages**: the sort is a single wide transformation (`sortByKey`) — no intermediate

Databricks Production Spark ML (2014)

Beyond sorting, Databricks published results showing Spark MLlib outperforming Mahout (MapReduce-based ML):

- **Logistic regression** on 1 TB: Spark 2 min vs. Mahout 10+ hours
- **Collaborative filtering** (ALS) on Netflix data: Spark 35 min vs. MR implementation 10 hours
- The speedup comes *primarily* from keeping the model and working set in RAM across iterations

Long-Term Impact

Lessons Learned & Discussion Questions

Key Lessons from Spark's Rise

- 1 **In-memory is a multiplier, not a panacea:**
Spark is 10–100 $\times$ faster on iterative workloads; for single-pass ETL over data larger than RAM, gains are smaller (2–5 $\times$)
- 2 **The lineage model scales elegantly:**
re-computing a lost partition from lineage is cheaper than maintaining 3 $\times$ replicated in-memory state
- 3 **Shuffle is the bottleneck:** both at GraySort and in production; minimising wide dependencies (shuffle) is the primary Spark tuning skill
- 4 **Benchmark marketing is real:** GraySort made Spark famous; it also hid cases where Spark's JVM memory overhead hurt performance (many small files, high-cardinality groupBy)

Discussion Questions (CLO3, CLO4)

- 1 A Spark job runs logistic regression over a 50 GB dataset on a 10-node cluster with 32 GB RAM/node. **(a)** Can the full dataset be cached at MEMORY_ONLY? **(b)** If not, which StorageLevel would you choose and why? **(c)** Estimate how many iterations are needed before the cached speedup (130 $\times$) pays for the initial caching cost (1 extra HDFS read). [CLO3 — Explain]
- 2 A groupByKey on an RDD with 1 billion records and 500 distinct keys produces 500 shuffle partitions. Identify **two** data skew scenarios that would cause OOM errors and propose a fix for each. [CLO4]

Live Exercise

Live Exercise — Spark RDD Lineage in PySpark

Task (25 minutes, pairs)

Run the following PySpark script locally (Spark local mode).

Goal: build and inspect an RDD lineage; observe lazy evaluation; compare cached vs. uncached timing.

Questions:

- 1 Call `rdd.toDebugString()` before and after `cache()` — what changes?
- 2 Time `rdd.count()` twice without `cache()`. Then add `.cache()` and time again. Report the speedup.
- 3 Change `reduceByKey` to `groupByKey`. Use the Spark UI (`localhost:4040`) to compare shuffle read/write bytes between the two.

PySpark Skeleton

```
from pyspark import SparkContext
import time

sc = SparkContext("local[4]", "RDDDemo")
sc.setLogLevel("WARN")

# Build lineage (lazy - no compute yet)
lines = sc.textFile("README.md")
words = lines.flatMap(lambda l: l.split())
pairs = words.map(lambda w: (w.lower(), 1))
counts = pairs.reduceByKey(lambda a, b: a+b)

# Inspect lineage BEFORE action
print(counts.toDebugString().decode())

# First action - triggers full DAG
t0 = time.time()
n = counts.count()
print(f"count={n}   time={time.time()-t0:.2f}s")

# Cache and re-run
counts.cache()
counts.count()           # warm up cache
```

References

Seminal Papers

- Zaharia, M., et al. (2010). Spark: Cluster computing with working sets. *Proc. 2nd USENIX HotCloud*.
- Zaharia, M., et al. (2012). Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. *Proc. 9th USENIX NSDI*, 15–28.

Textbooks [SA], [TE], [TW]

- **[SA]** Acharya, S., & Chellappan, S. (2015). *Big Data and Analytics* (Ch. 6 — Apache Spark). Wiley.
- **[TE]** Erl, T., Khattak, W., & Buhler, P. (2016). *Big Data Fundamentals* (Ch. 8 — In-Memory Processing). Prentice Hall.
- **[TW]** White, T. (2015). *Hadoop: The Definitive Guide*, 4th ed. (Ch. 19 — Spark). O'Reilly.

Case Study Source

- Databricks Engineering Blog. (2012, November). *Spark Officially Sets a New Record in Large Scale*

Key Takeaways

- Zaharia et al. (2012) showed that **lineage is a superior fault-tolerance mechanism** for iterative in-memory workloads — cheaper than replication, and it enables lazy evaluation
- The **narrow / wide dependency** distinction directly determines Spark's stage structure: minimising wide dependencies is the single most important Spark performance tuning heuristic
- Databricks' GraySort result (32× better efficiency per node) proved that **algorithm + hardware co-design** — sort-based shuffle on SSDs — matters as much as in-memory caching
- RDDs are now largely replaced by **DataFrames and Datasets** (since Spark 2.0), but every Spark DataFrame operation compiles down to an RDD

Quote of the Week

“RDDs can express many current cluster programming models — MapReduce, DryadLINQ, Pregel, HaLoop — as Spark libraries.”

— Zaharia et al. (2012)

Part 1 Preview — Week 7

HiveQL, Partitioning, Bucketing & Presto

- HiveQL execution plan walkthrough
- Partitioning vs. bucketing trade-offs
- Presto/Trino federation model
- Cost-based optimiser and statistics